

PROJECT KEYSTONE

Field Service Management Platform

Project Documentation

1. Overview

KEYSTONE is a field service management platform that replaces spreadsheet- and messaging-based coordination with a single system of record. It covers customer requests, work orders, assignments, field updates, SLA monitoring and reporting.

Objectives

- Track every request as a work order.
- Give dispatchers assignment and board visibility.
- Give technicians a responsive field workflow.
- Give managers operational and SLA visibility.
- Give customers self-service status tracking.
- Enforce security and lifecycle rules on the server.

Main Domain

Customer, Site, User, WorkOrder, WorkOrderStatusHistory, Part, PartUsage and TimeLog.

Work-Order Lifecycle

NEW → ASSIGNED → IN_PROGRESS → ON_HOLD → IN_PROGRESS → COMPLETED → CLOSED.

Cancellation is supported before terminal completion. Illegal transitions are rejected by the service layer, and each valid status change creates an append-only history record.

2. Technology Stack

Layer	Technologies
Backend	Java 21, Spring Boot, Spring Security, JWT, Spring Data JPA, Hibernate, PostgreSQL, Flyway, Bean Validation, springdoc-openapi
Frontend	React, TypeScript, Vite
Deployment	Docker Compose for local development; prepared for cloud backend / frontend / database deployment

3. Architecture

KEYSTONE follows a layered architecture. Controllers stay thin; business rules and transactions live in the service layer, and DTOs form the API boundary.

KEYSTONE — Layered Architecture

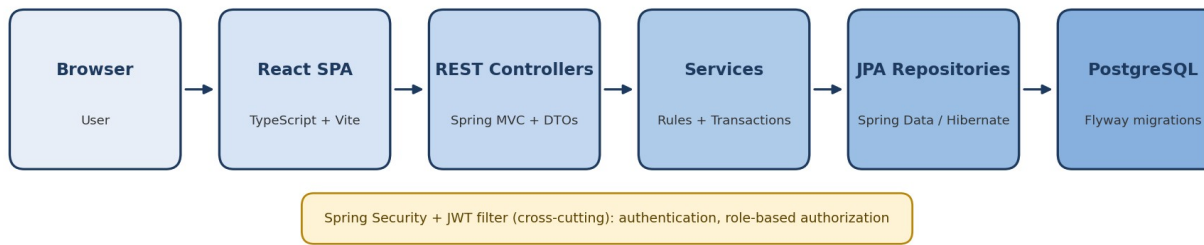


Figure 1 – Layered architecture

Security

- Stateless JWT authentication, applied through Spring Security and a JWT filter.
- Passwords encoded with BCrypt.
- Role-based authorization on protected endpoints.
- Customer and technician access scoped to the data they are allowed to operate on.

Data Integrity

- Part usage and stock decrement occur in a single transaction.
- Stock cannot go below zero.
- Status history is append-only.

Representative API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate and receive JWT
GET/POST	/api/customers	List / create customers
GET/POST	/api/customers/{id}/sites	List / create customer sites
GET/POST	/api/work-orders	List / create work orders
PUT	/api/work-orders/{id}	Update a work order
POST	/api/work-orders/{id}/assign	Assign a technician
POST	/api/work-orders/{id}/status	Change lifecycle status
POST	/api/work-orders/{id}/parts	Record part usage
POST	/api/work-orders/{id}/time	Log time
GET	/api/reports/summary	Operational summary report

4. Conclusion

KEYSTONE demonstrates a complete Java full-stack architecture: a Spring Boot API, relational persistence, JWT security, a React UI and a governed operational workflow. The key challenge — coordinating authorization, state transitions, audit history and transactions without placing business rules in the React interface — is addressed by keeping all rules in the service layer. The project is structured for clean local setup and cloud deployment.